Unit-3
Basic Behavior modeling

Interactions

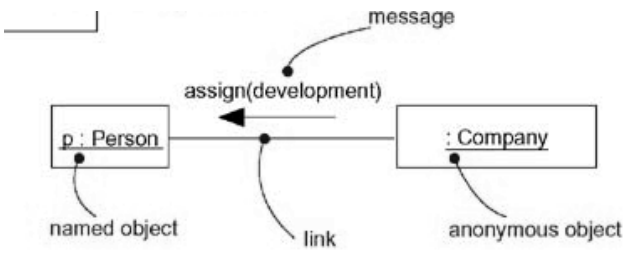
- An interaction is a behavior that comprises a **set of messages exchanged among a set of objects** within a context to accomplish a purpose.
- A **message is a specification of a communication between objects** that conveys information.

Objects and Roles

- We can use interactions to **visualize, specify, construct, and document the semantics of a class/object**
- We may find an **interaction wherever objects are linked to one another**.
- We'll **find interactions in the collaboration of objects that exist in the context of your model.**
- The objects that participate in an interaction are either **concrete things or prototypical things**.
- As **a concrete thing**, an object represents something in the **real world.**
- As a **prototypical thing**, p might represent any **instance of Person**.

Links

- A **link is a semantic connection among objects.**
- Wherever a **class/object has an association to another class**/object, there **may be a link between the instances of the two classes**/objects.
- Wherever there is a **link between two objects, one object can send a message to the other object**



Links and Associations

- We can adorn the appropriate end of the link with any of the following standard stereotypes

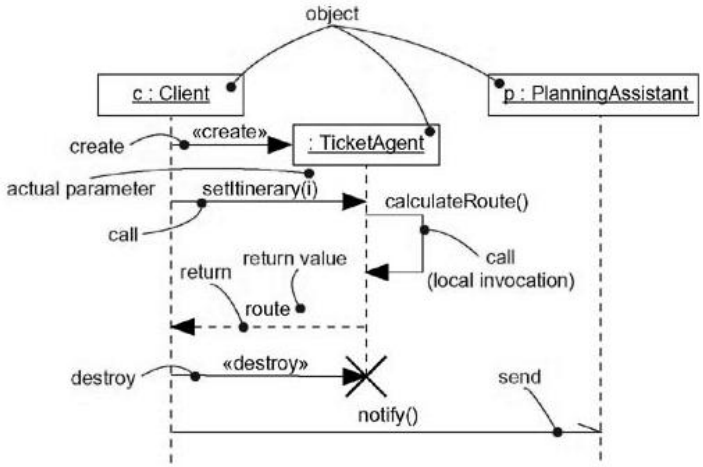
association	Specifies that the corresponding object is visible by association
self	Specifies that the corresponding object is self visible.
global	Specifies that the corresponding object is visible because it is in an global scope
Local	Specifies that the corresponding object is visible because it is in a local scope

Messages

- A **message is the specification of a communication among objects** that **conveys information**.
- In the UML, you can model several kinds of actions

Return	Returns a value to the object
Send	Sends a signal to an object
Create	Creates an object
Destroy	Destroys an object; an object may commit by destroying itself

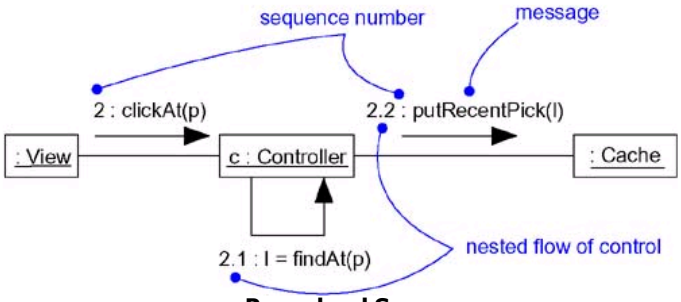
- The UML provides a visual distinction among these kinds of messages, as follows



Messages

Sequencing

- When an **object passes a message to another object the receiving object might in turn send a message to another object,** which might send a message to yet a different object, and so on.
- This **stream of messages forms a sequence Any sequence must have a beginning.**
- **Messages are ordered in sequence by time.**
- Most commonly, you can specify a **procedural or nested flow of control**, rendered using a filled solid arrowhead



Procedural Sequence

Creation, Modification, and Destruction

In **UML (Unified Modeling Language)**, **Creation, Modification, and Destruction** of objects are key aspects modeled in **interaction diagrams** (especially in **sequence diagrams** and **communication diagrams**).

New	Indicates that the instance (object or link) is **created** during the execution of the **enclosing interaction**.
Destroyed	Specifies that the instance is **destroyed before** the completion of the **enclosing interaction**.
Transient	Specifies that the instance is **both created and destroyed** during the **same interaction**.

Representation

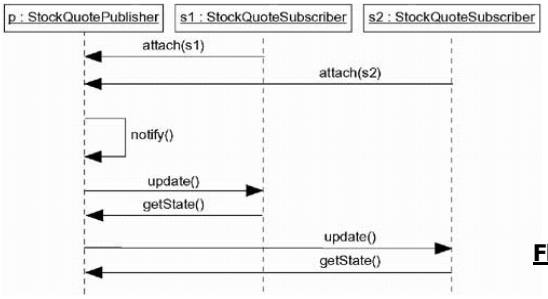
- When you model an **interaction, you typically include both objects and messages**
- We can visualize those **objects and messages involved in an interaction in two ways**
  - By emphasizing the **time ordering of its messages**
  - by emphasizing **the structural organization of the objects** that send and receive messages.
- In the UML, **the first kind of representation is called a sequence diagram**
- The second kind of **representation is called a collaboration diagram**
- **Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams**

Common Modeling Techniques

Modeling a Flow of Control by time ordering

- **Use Sequence Diagrams**
  - Represents the time-ordered interaction between objects.
  - Shows how control passes from one object to another.
- **Identify the Participating Objects (Lifelines)**
  - Each object involved is shown as a vertical dashed line (lifeline).
  - Place objects horizontally across the top of the diagram.
- **Arrange Messages in Time Order (Top to Bottom)**
  - Each interaction (message call) is represented by a horizontal arrow from one object to another.
  - Messages are ordered vertically to show time progression.
- **Show Message Types**
  - **Synchronous messages** (calls)
  - **Asynchronous messages** (signals)
  - **Return messages** (dashed lines)
- **Represent Creation and Destruction of Objects**
  - Object creation: <<create>> message, with lifeline starting at that point.
  - Object destruction: <<destroy>> message, marked with an X.

- This figure is an example of a sequence diagram, which emphasizes the time order of messages.

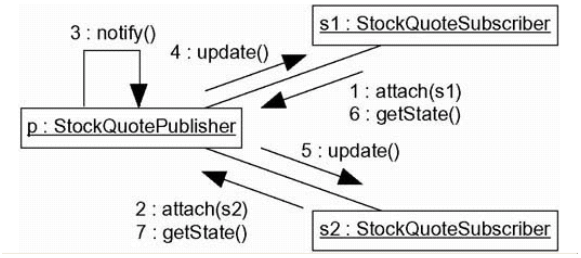


Flow of Control by Time

Flow of Control by structural Organization

- Use Collaboration (Communication) Diagrams
  - Show **interactions organized by object links** (structural relationships), not time.
  - Emphasizes which **objects communicate and how they are connected.**
- Focus on Object Links
  - O**bjects (instances) are connected through links based on structural associations.**
  - **These links define possible communication pat**hs.
- Message Flow Along Structural Paths
  - Messages **are sent along these links**; **messages are labeled** with **sequence numbers to indicate order.**
- Use Sequence Numbers to Indicate Control Flow
  - Example: 1, 1.1, 1.2, 2, etc., to show nested or sequential message flow.

This figure is semantically equivalent to the previous one, but it is drawn as a collaboration diagram, which emphasizes the structural organization of the objects.



Interaction Diagrams

Terms and Concepts:

Def : An interaction diagram shows an interaction, consisting of a set of objects and their relationships, including the messages that may be dispatched among them.

- **A sequence diagram is an interaction diagram that emphasizes the time ordering of messages. Graphically, a sequence diagram is a table that shows objects arranged along the X axis and messages, ordered in increasing time, along the Y axis.**

- **A Collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages. Graphically, a Collaboration diagram is a collection of vertices and arcs.**

Contents:

Interaction diagrams commonly contain

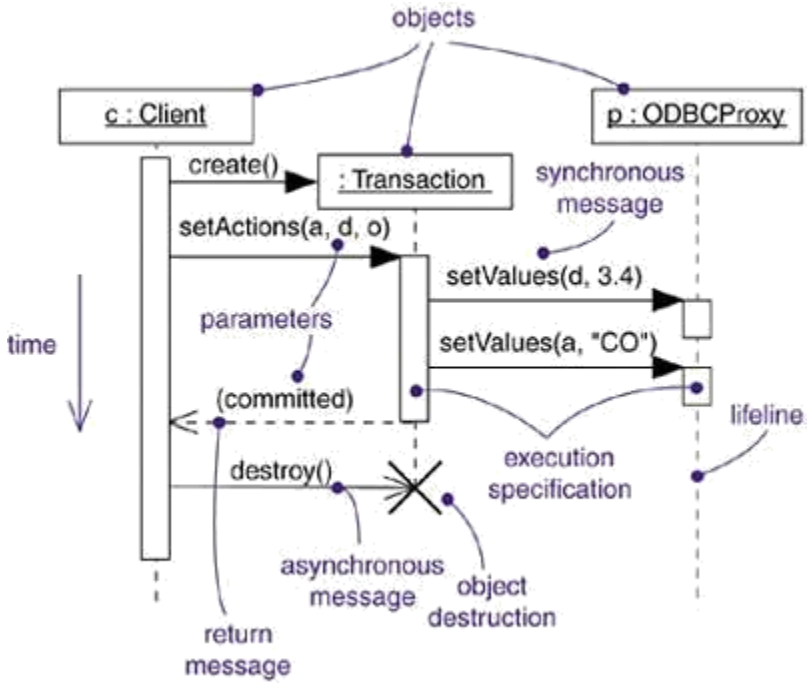
- **Objects**
- **links**
- **Messages**
- Like all other diagrams, interaction diagrams may contain **notes and constraints**.

Sequence Diagrams:

A sequence diagram emphasizes the time ordering of messages. sequence diagram by first placing the objects or roles that participate in the interaction at the top of your diagram, across the horizontal axis.

Sequence diagrams have two features that distinguish them from Collaboration diagrams.

First, there is *the lifeline.* An object lifeline is *the vertical dashed* line that represents the *existence of an object over a period of time*.



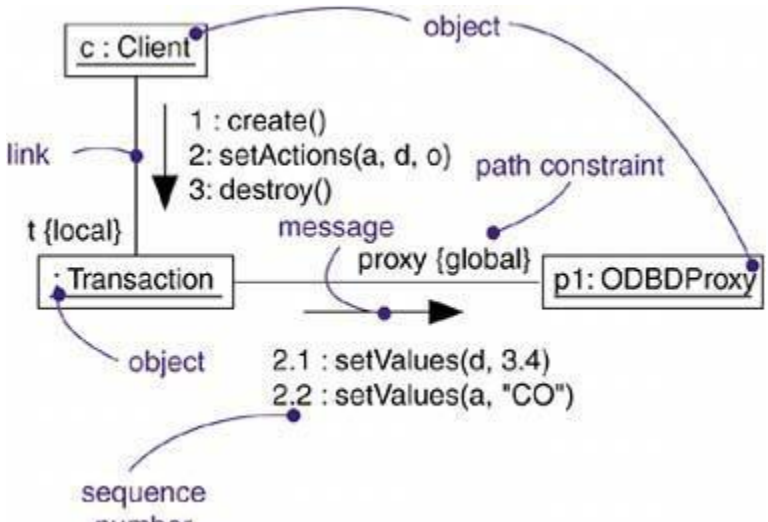
Second, there is the *focus of control*. The focus of control is a tall*, thin rectangle that shows the period of time during which an object is performing an action*, either directly or through a subordinate procedure. The *top of the rectangle is aligned with the start of the action; t*he *bottom is aligned with its completion* .

Collaboration Diagrams:

A Collaboration diagram emphasizes the organization of the objects that participate in an interaction.

- **Collaboration diagram by first placing the objects that participate in the interaction as the vertices in a graph.**
- Next, ***you render the links that connect these objects as the arcs of this graph***.
- The **links may have role names to identify them.**

with the messages that objects send and receive. This **gives the reader a clear visual** to the ***flow of control in the context of the structural organization of objects that collaborate***.



Collaboration diagram have **two features** that distinguish them from sequence diagrams.

First, there is the path. You render a path corresponding to an association. You also render paths corresponding to local variables, parameters, global variables, and self access. A path represents a source of knowledge to an object.

Second, there is the sequence number. To indicate the time order of a message, you prefix the message with a number (starting with the message numbered 1), increasing monotonically for each new message in the flow of control (2, 3, and so on). To show nesting, you use Dewey decimal numbering (1 is the first message, which contains message 1.1 and message 1.2, and so on). You can show nesting to an arbitrary depth. Note also that, along the same link, you can show many messages (possibly being sent from different directions), and each will have a unique sequence number.

Semantic Equivalence:

Because they both derive from the same information in the UML's metamodel, sequence diagrams and communication diagrams are semantically equivalent. As a result, you can take a diagram in one form and convert it to the other without any loss of information, as you can see in the previous two figures, which are semantically equivalent. However, this does not mean that both diagrams will explicitly visualize the same information.

Common Uses:

1. **To model flows of control by time ordering**

- **Sequence Diagrams** focus on the **temporal sequence** of messages (calls, returns, signals).
- Time progresses **vertically from top to bottom**.
- Each **lifeline** represents an object or participant.
- **Messages** are arrows between lifelines, ordered by time.
- You can model **synchronous, asynchronous, and return messages**.

2. **To model flows of control by organization**

- Focuses on the **structure** (which objects interact), rather than time.
- Shows **how objects are linked** and how they **collaborate**.
- Emphasizes the **organizational relationships** between participants.
- Messages are **numbered** (e.g., 1, 1.1, 2) to indicate **sequence**.
- Good for understanding **how responsibilities are distributed**.

/??>

Common Modeling Techniques:

Modeling Flows of Control by Time Ordering

1. Use Sequence Diagrams

- Sequence diagrams are the **primary tool** in UML to model control flow over time.
- They show how **objects interact in a particular order** during a scenario.
- The **vertical axis represents time** (top to bottom).
- The **horizontal axis represents objects or participants**.

🔷 **2. Identify Key Objects and Actors**

- Determine all the **objects, components, or actors** involved in the interaction.
- Represent them as **lifelines** across the top of the diagram.

🔷 **3. Model Lifelines**

- Each object is shown with a **dashed vertical line (lifeline)**.
- The lifeline shows the object's **existence over time**.

🔷 **4. Add Messages in Time Order**

- Show **messages** (method calls, signals, returns) as arrows between lifelines.
- Messages are **ordered from top to bottom** to reflect time sequence.
- Use **solid arrows** for synchronous calls and **open arrows** for asynchronous messages.

🔷 **5. Use Activation Bars**

- Activation bars (thin rectangles) indicate when an object is **performing an action**.
- These visually represent the **duration** of execution for each operation.

🔷 **6. Use Return Messages (optional)**

- Dotted arrows indicate **return messages** or responses.
- Not always necessary but help clarify result flow.

Modeling Flows of Control by Organization

1. **Use Communication Diagrams**

- **Communication Diagrams** are ideal for modeling how **objects collaborate** based on their **organizational relationships**.
- Focus is on **which objects interact**, rather than the exact timing.

---

2. Identify Participating Objects and Roles

- Identify the **objects/components** and the **roles** they play in the scenario.
- Place these objects spatially to reflect **logical or organizational structure** (e.g., layered architecture).

---

3. Show Links Between Objects

- Use **associations (solid lines)** between objects to represent that they can **communicate**.
- These links represent **structural relationships**, similar to object references in code.

---

4. Add Numbered Messages

- Indicate **messages** passed between objects using **arrows** labeled with **sequence numbers** (e.g., 1, 1.1, 2).
- Numbers show the **order of message flow** in the interaction.

---

5. Use Message Sequencing and Nesting

- Use **nested numbering** (e.g., 1, 1.1, 1.2) to show **sub-messages** or message calls inside other calls.
- This reflects **hierarchical control flow** within the organization.

Forward and Reverse Engineering:

Forward engineering (the creation of code from a model) is possible for both sequence and communication diagrams, especially if the context of the diagram is an operation. For example, using the previous communication diagram, a reasonably clever forward engineering tool could generate the following Java code for the operation register, attached to the Student class.

Reverse engineering (the creation of a model from code) is also possible for both sequence and communication diagrams, especially if the context of the code is the body of an operation. Segments of the previous diagram could have been produced by a tool from a prototypical execution of the register operation.

Use Cases

Usecase definition:

A Use Case in UML (Unified Modeling Language) is a a **set of sequences of actions, description of how a user (actor) interacts with a system to achieve a specific goal. It represents a functional requirement of the system.**

Key Points:

- Describes **user–system interaction**.
- Focuses on the **goal or functionality** from the user's perspective.
- Each use case represents a **single function** of the system.
- Used to **identify, clarify, and organize** system requirements.

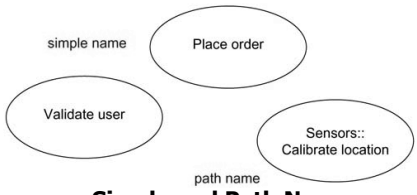
Example:

For an **Online Banking System**,
some use cases are:

- *Login to Account*
- *Transfer Money*
- *Check Account Balance*
- *Pay Bills*

Names

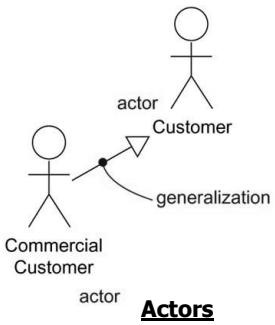
- **Every use case must have a name** that distinguishes it from other use cases.
- **A name is a textual string**.
- That name alone is known as **a simple name**; **a path name(qualified name)** is the use case name prefixed by the name of the package in which that use case lives.
- A use case is typically drawn showing only its name



Simple and Path Names

Use Cases and Actors

- An **actor represents set of roles** that **users of use cases play when interacting with these use cases**.
- **Actors may be connected to use cases only by association.**
- An **association between an actor and a use case indicates that the actor and the use case communicate with one another**, each one possibly **sending and receiving messages.**



Actors

Use Cases and Flow of Events

- Describe **functional behavior** of a system from the **user's point of view**.
- Represent **interactions between actors and the system**.
- Each use case defines a **specific goal** or **service** provided by the system.
- Help in **identifying system requirements** clearly.
- Shown in **Use Case Diagrams** using ovals.
- Connected to **actors** that represent users or external systems.
- Can include relationships like **include**, **extend** between use cases.

For example, in the context of an ATM system, you might describe the use case ValidateUser in the following way:

Flow of Events:

1. Describes the **detailed sequence of actions within a use case.**
2. Explains **how the system and actors interact step by step.**
3. Divided into:
    - **Basic Flow (Main Flow): Normal successful path.**
    - **Alternative Flow: Optional or conditional paths.**
    - **Exception Flow: Error or failure handling.**
4. Written **in text form, not shown in the diagram**.
5. Helps **developers understand the logic and behavior of the system**.
6. Acts as a **bridge between requirements and design**.

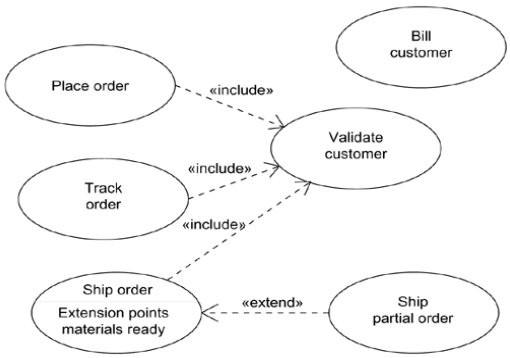
➢ **Exceptional flow of events:**

- Describes **what happens when something goes wrong** during the normal execution of a use case.
- Represents **unexpected or error conditions** that prevent the system from completing the main flow.
- Helps in defining **how the system should respond** to such exceptions.
- Usually follows after the **Basic Flow** and **Alternative Flow** descriptions in a use case.
- Can include **system errors, invalid inputs, network failures, or missing data**.
- Provides a **recovery or termination path** to bring the system back to a stable state.
- Improves the **reliability and robustness** of the system design.

Common Modeling Techniques

Modeling the Behavior of an Element

- **Identify the element whose behavior is to be modeled**
  - Determine whether it's a system or subsystem.
- **Define its responsibilities and operations**
  - Specify what the element should do (functions, responses, actions).
- **Specify use cases that represent the behavior**
  - Create *use cases* showing how actors interact with the element to achieve goals.
- **Model interactions with external actors**
  - Show *actors* (users, systems, or devices) that communicate with the element.
- **Describe the flow of events**
  - Write *main, alternate, and exceptional* flows for each use case.
- **Show relationships between use cases**
  - Use <<include>>, <<extend>>, and *generalization* to capture behavioral variations.



Modeling the Behavior of an Element

Use Case Diagram

- A **use case diagram is a diagram that shows a set of use cases and actors and their relationships.**

Contents

- Use case diagrams commonly contain
    - **Use cases**
    - **Actors**
    - **Dependency, generalization, and association relationships.**

Common Uses

- We apply use case diagrams to model the **static use case view of a system**.
- This view **supports the behavior of a system.**
- When you model the static use case view of a system, you'll typically apply use case diagrams in one of two ways.
    - **To model the context of a system**
    - **To model the requirements of a system**

Common Modeling Techniques

Modeling the Context of a System

1. Modeling the Context of a System
Meaning:
Modeling the *context of a system* means defining the **boundaries** of the system and identifying how it **interacts with external entities** (people, devices, or other systems).

Purpose:

To understand what lies **inside** the system (what it will do) and what lies **outside** (who or what interacts with it).

- Identify **actors** (external users or systems) that communicate with your system.
- Identify **system boundaries** (what functions belong to the system).
- Use a **Use Case Diagram** to show:
    - The system as a single rectangle (the boundary).
    - Actors outside it.
    - Use cases inside it that represent system functions.
- Show **communication links** (associations) between actors and use cases.

Example:
In an **ATM system**, the context includes:

- **Actors:** Customer, Bank Database.
- **System:** ATM Machine.
- **Interactions:** Withdraw Cash, Check Balance, Deposit Money.

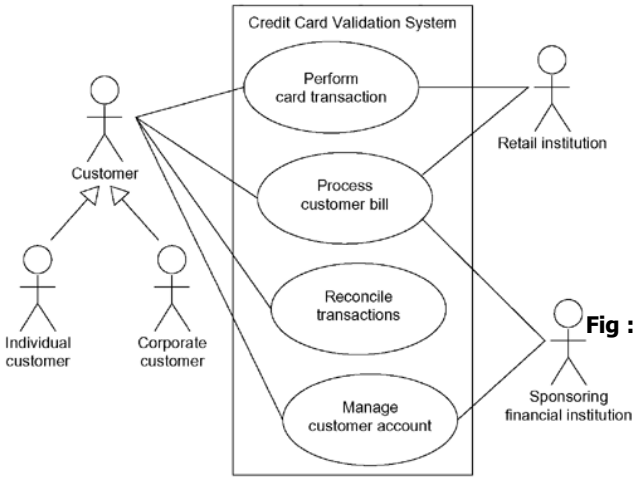


Fig : Modeling the Context of a System

Modeling the Requirements of a System

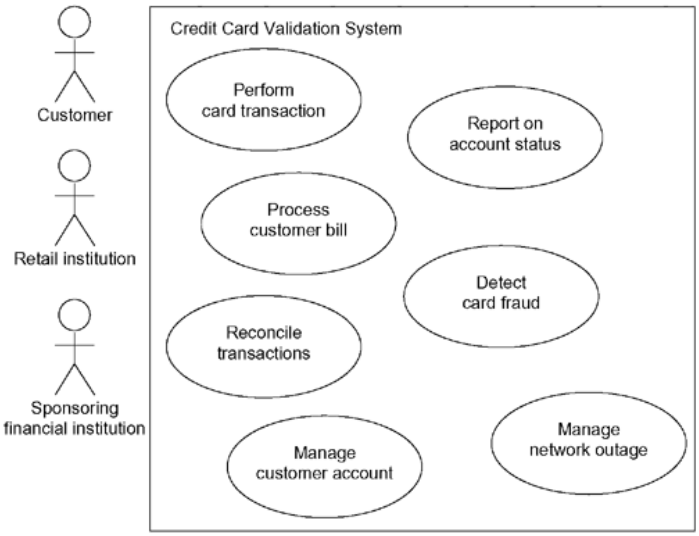
Meaning:
Modeling the *requirements of a system* means describing **what the system should do** — its **functional** and **non-functional** requirements — from the user's point of view.

Purpose:
To capture and represent the **expected behavior and functionalities** of the system that satisfy user needs.

- Identify **use cases** that describe system functionalities.
- Write **detailed use case descriptions** (with flow of events, pre/post conditions).
- Use **UML Use Case Diagrams** to visually show functional requirements.
- Use **activity diagrams** to represent workflow of a use case.
- Use **requirements traceability** to connect use cases to design elements later.

Example:
For the **ATM system**, requirements include:

- The system must allow customers to withdraw cash.
- The system must verify PIN before transactions.
- The system must display account balance on request.



Modeling the Requirements of a System

Forward and Reverse Engineering

- **Forward engineering** is the process of transforming a model into code through a mapping to an implementation language.
- A use case diagram can be forward engineered to form tests for the element to which it applies.
- Each use case in a use case diagram specifies a flow of events and these flows specify how the element is expected to behave

- **Reverse engineering** is the process of transforming code into a model through a mapping from a specific implementation language.
- The UML's use case diagrams simply give you a standard and expressive language in which to state what you discover.

Activity Diagrams

- An **activity diagram shows the flow from activity to activity.**
- Actions encompass **calling another operation, sending a signal, creating or destroying an object**, or some pure computation, such as evaluating an expression.
- **Graphically, an activity diagram is a collection of vertices and arcs.**

Contents

- Activity diagrams commonly contain
  - **Activity states and action states**
  - **Transitions**
  - **Objects**
- Like all other diagrams, activity diagrams may contain notes and constraints.

Action States and Activity States

Definition:
An **action state** represents a simple, *instantaneous action* that cannot be interrupted once started.

Key Points:
- Takes **negligible time** to complete.
- Represents **atomic actions** (executed as a single step).
- No other event can interrupt it while executing.
- Usually written with a short name like "calculate total" or "send message".

Example:
validate input, increment counter, display message

Diagram Notation:
A **rounded rectangle** (with slightly curved edges).

---

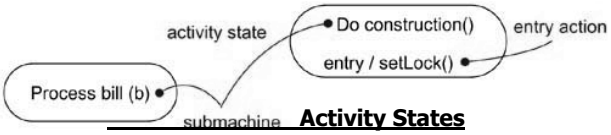
2. Activity States

Definition:

An **activity state** represents a **non-instantaneous** operation that may take **time to complete** and **can be interrupted** by other events.

Key Points:
- Represents **longer-running behavior**.
- May involve **multiple actions** or **internal states**.
- Can have **entry**, **do**, and **exit** activities.
- Control can move to other states based on events while the activity is ongoing.

Example:
process payment, download file, print document
Diagram Notation:
Also shown as a **rounded rectangle**, but semantically represents a continuous activity, not a single-step action.



Activity States

Transitions

- When the action or activity of a state completes**, flow of control passes immediately to the next action or activity state.**
- We **specify this flow by using transitions to show the path from one action or activity** state to the next action or activity state.
- In the UML, you represent **a transition as a simple directed line**

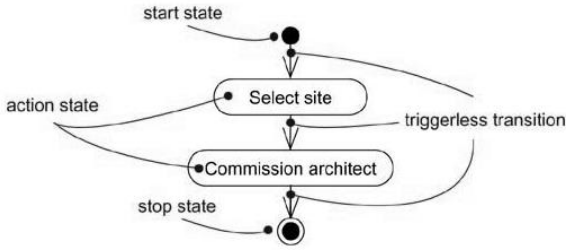
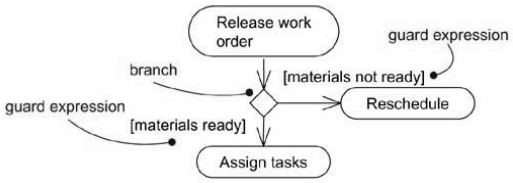


Fig : Triggerless Transitions

Branching

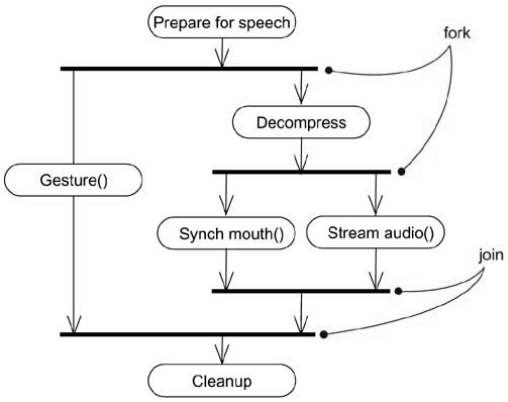
- As in a flowchart, you can include a branch, **which specifies alternate paths taken based on some Boolean expression.**
- We represent a branch as a diamond. A branch may have **one incoming transition and two or more outgoing ones.**
- On each outgoing transition, you place a Boolean expression, which is evaluated only once on entering the branch.
- On **each outgoing transition, you place a Boolean expression**, which is evaluated only once on entering the branch.



Branching

Forking and Joining

- When we are modeling workflows of business processes—we might encounter flows that are concurrent.
- In the UML, you use a synchronization bar to specify the forking and joining of these parallel flows of control. A synchronization bar is rendered as a thick horizontal or vertical line.
- **Fork** represents the splitting of a single flow of control into two or more concurrent flows of control
- A fork may have one incoming transition and two or more outgoing transitions, each of which represents an independent flow of control.

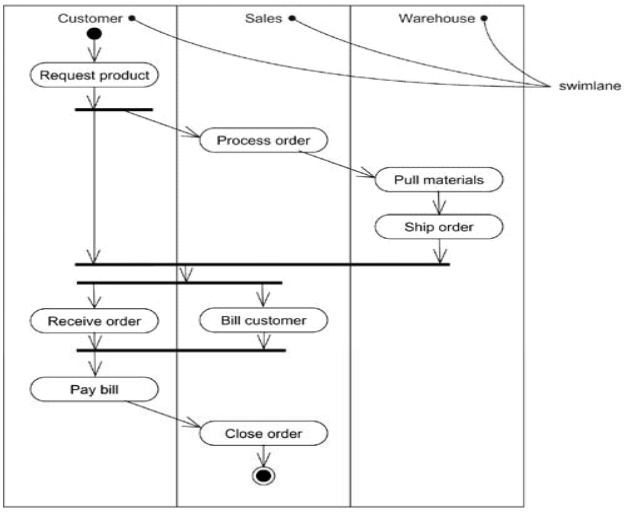


Forking and Joining

- **A Join represents the synchronization of two or more concurrent flows of control.**
- A join may have two or more incoming transitions and one outgoing transition.
- Above the join, the activities associated with each of these paths continues in parallel.

Swimlanes

- We'll find it useful, especially when you are modeling workflows of business processes, to partition the activity **states on an activity diagram into groups**, **each group representing the business organization responsible for those activities.**
- In the UML, each group is called a swimlane because, visually, each group is divided from its neighbor by a vertical solid line
- **Each swimlane has a name unique within its diagram**.
- Each swimlane represents a high-level responsibility for part of the overall activity of an activity diagram, and each swimlane may eventually be implemented by one or more classes.



Swimlanes

Common Uses

- We use activity diagrams to model the dynamic aspects of a system
- These dynamic aspects may involve the activity of any kind of abstraction in any view of a system's architecture, including classes, interfaces, components, and nodes.
- When you model the dynamic aspects of a system, we'll typically use activity diagrams in two ways.
    - **To model a workflow**
    - **To model an operation**

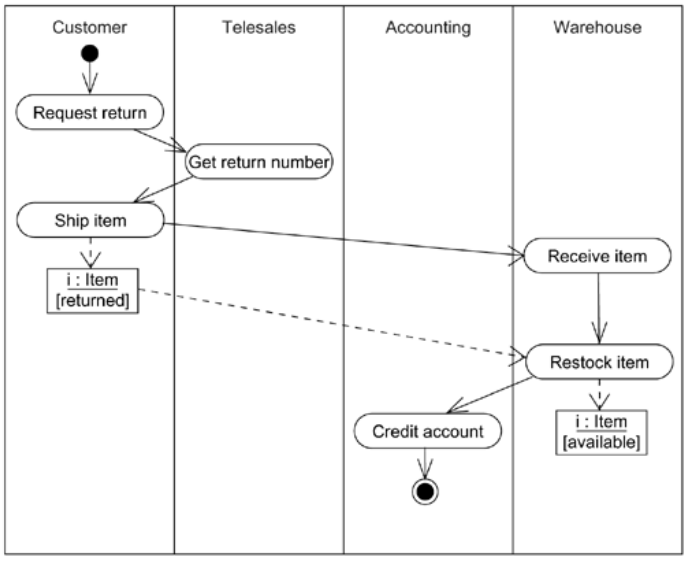
Common Modeling Techniques

Modeling a Workflow

Purpose:
To represent the **sequence of activities and flow of control i**n a business process or system workflow.

Key Points:

- Used to describe **how a process or system behaves** as **a series of activities**.
- Focuses on **business-level processes** and **data flow** between activities.
- Shows **the order in which activities are performed** and the conditions for transitions.
- Activities may be performed **sequentially or concurrently**.
- Helps in **analyzing, optimizing, and automating** workflows.
- Involves **decision points**, **forks**, **joins**, and **swimlanes** to represent **responsibilities of actor**s or departments.
- Useful in **business process modeling**, and **system workflow design**.



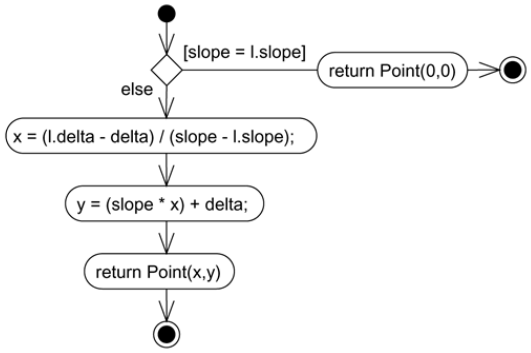
Modeling a Workflow

Modeling an Operation

Purpose:
To describe the **detailed behavior** or **internal logic** of a single operation or method within a class.

Key Points:

- Used to model **algorithmic flow** or **method implementation** in UML.
- Focuses on **how a particular operation is executed**, step-by-step.
- Represents **control flow, decisions, loops, and actions** within the operation.
- Helps in **understanding, documenting, and implementing** the operation logic.
- Useful for **specifying internal behavior** of classes or system components.
  -
-



Modeling an Operation

Forward and Reverse Engineering

- **Forward engineering** (the creation of code from a model) is possible for activity diagrams, especially if the context of the diagram is an operation.

- **Reverse engineering** (the creation of a model from code) is also possible for activity diagrams, especially if the context of the code is the body of an operation.
- In particular, the previous diagram could have been generated from the implementation of the class Line.